

Note-set demo script — specification-driven development without the waterfall

A single-class (75-minute) live demo: Claude Code maintains a small set of markdown study notes under two written specifications, and the class watches the spec-implementation conformance relationship survive change in both directions. It pairs with the specification/implementation lectures (`../../../../weeks-01-03/lecture-notes/lecture-04-prompting-and-spec-driven-development.md`), makes the S/R essay executable (`../../../../software-engineering/specification-implementation.md`), and is Project 0 in miniature (`../../../../weeks-01-03/exercises/project-0-pkb-kickoff.md`; house style per `../../../../weeks-01-03/student-repo/pkb-example/`). Paths are relative to this folder's location in the course repository.

The critique this demo answers: “specification-driven development” is sometimes pitched as waterfall in disguise — finish a spec, feed it to an agent, receive an implementation, one-way flow. The demo makes five counter claims *observable*:

1. A specification is a continuously maintained document of intent.
2. A specification is an abstraction of the implementation.
3. Spec-implementation conformance is an invariant maintained while **both** sides evolve.
4. There are multiple specifications, and they must be internally and mutually consistent.
5. Information flows both ways: implementation work refines the specs.

A caution that governs the whole script. Never depend on the agent making one specific move. Agent behavior varies run to run and version to version; the *Expected* notes below describe the typical shape, and every segment carries an *If it goes differently* recovery. Rehearse the full demo at least once before class; the rehearsal, not this script, is the source of truth for what your setup actually does. Capture screenshots at the `[fallback capture]` markers during rehearsal so a misbehaving live run never strands a teaching point.

Folder layout

- **starter/** — the demo's t=0 state: two draft specifications (`note-set-conops.md`, `note-format-spec.md`) and a `CLAUDE.md` of working rules. This is the **only** directory copied into the live demo repo.
- **sample-inputs/** — the professor's paste material: initial content for the add-with-content operation, the deliberately non-conformant note used in Segment 5, and optional Segment 7 content. Kept **outside** the demo repo so the agent cannot read the violations it is about to be tested on.
- **completed/** — a representative end state: specs at their final versions, three conformant notes, the operations document, the index, and a reference `check_notes.py`. Every live run differs — different gap lists, different summary wording — and that is expected; this folder shows the approximate destination, and its `check_notes.py` is the Segment 6 live failure fallback. `CLAUDE.md` is byte-identical in **starter/** and **completed/**: the working rules never change during the demo, which is itself worth a sentence in class.

Three deliberate gaps are seeded in the starter specs; Segment 2 exists to let the agent find them. For your eyes (do not reveal before Segment 2):

- **Gap A (between the specs):** the format spec's preamble says “every markdown file in the note set MUST satisfy the rules below,” and the ConOps says `index.md` is a markdown file in the repo — but `index.md` has no front-matter and cannot satisfy R1/R2. Neither document says whether the index is a note.
- **Gap B (inside the format spec):** R2 requires a front-matter `title` and R3 requires an H1, but no rule says they must agree.
- **Gap D (spec vs. execution):** O1 and O2 supply only a *title*, and no rule anywhere derives a *filename* from it. The agent literally cannot execute “add blank note” without inventing a convention.

(The date format is deliberately *not* a gap — R2 pins ISO-8601 from v0.1 — so that in Segment 5 an original rule and a refinement-added clause each catch a violation on the same note.)

Before class — setup checklist

1. Create the demo repo from **starter/** only, and make the initial commit:

```
rm -rf ~/note-set-demo
cp -R <path-to-this-demo>/starter ~/note-set-demo
cd ~/note-set-demo
git init
git add -A
git commit -m "demo start: draft specs only"
```

2. Verify plain python3 works (Segment 6 needs it, stdlib only):

```
python3 --version
```

3. Copy the reference checker somewhere **outside** the demo repo, as the Segment 6 fallback:

```
mkdir -p ~/note-set-demo-fallback
cp <path-to-this-demo>/completed/check_notes.py ~/note-set-demo-fallback/
```

4. Open the three **sample-inputs/** files in a **plain-text** editor window (not a markdown-aware editor — some will silently “fix” the seeded heading violations on paste, which destroys Segment 5).
5. Start Claude Code in **~/note-set-demo** in its default permission mode; confirm it picks up **CLAUDE.md**. Two terminals, large font: one for Claude Code, one for **cat/git/python3**.
6. Note on dates: sample inputs carry fixed dates; notes created live get demo-day dates. The checker validates date *format*, not value — nothing needs “fixing” mid-demo.
7. Rehearse the full run once. Capture screenshots at every [**fallback capture**] marker. Then reset for class by repeating step 1.

Segment 0 — Framing: the invariant, not the waterfall (~4 min)

No agent yet.

Do (shell): `ls ~/note-set-demo` — two specifications and a **CLAUDE.md**. No implementation.

Say: the critique — spec-driven development pitched as “finish the spec, an agent derives the implementation” is waterfall reborn: one-way flow from intent to spec to code. Today’s counter-picture is the S/R essay made executable: a specification *S* and a realization *R* connected by a conformance relation, and the relation is maintained as an *invariant while both ends move*. State the five claims above; promise the class one piece of evidence for each within the hour.

Say (setting the hook): “These specs are drafts, and at least one thing in them is wrong. We will not find it by staring at them.”

Segment 1 — Tour of the drafts (~7 min)

Do (shell): `cat note-set-conops.md`.

Say: this is ConOps altitude (lecture 04): a non-programmer could read it and confirm “yes, that’s the tool I want.” Walk O1–O3 and their pre/postconditions — contracts on *operations*, not code. Point at **Version: 0.1 (draft)**: specifications carry versions, like code.

Do (shell): `cat note-format-spec.md`.

Say: this one is normative — RFC-2119 MUST/SHOULD, and the rules are *numbered*. Read R1–R6 aloud; it takes under a minute, which is the point: a format spec a class can hold in its head. “The numbering is so that a conformance report — from an agent *or a tool* — can cite rules by ID. Remember the PKB spec’s checklist, ‘written to be script-generable later’? That was foreshadowing.”

Do (shell): `cat CLAUDE.md`.

Say: working rules for the agent, each engineered to make something visible today. Flag rule 2 (spec changes need approval — “matters in ten minutes”) and rule 4 (report violations before fixing — “matters in about forty”). Do **not** reveal the seeded gaps.

[fallback capture] — both specs on screen.

Segment 2 — Planning: the specs push back (~13 min) — the centerpiece

Do: in Claude Code, switch to **plan mode** (Shift+Tab cycles modes; UI varies by version — say so). Plan mode means read-only: the agent can explore and propose, not touch.

Do: prompt:

Read note-set-conops.md and note-format-spec.md carefully. I want to set up this note set so it fully realizes the ConOps: create the notes directory and the index, and document each agent operation — add blank note, add note with initial content, remove note, check conformance — with preconditions and postconditions in a new file note-set-operations.md. Before proposing a plan, list every place where the two specifications are ambiguous, incomplete, or inconsistent with each other — anything that would force you to guess during implementation. Number the issues and ask me to resolve them before you plan.

Expected: visible Read calls on both specs, then a numbered issue list containing at least the three seeded gaps: is `index.md` a note; must the H1 match the `title` field; what filename does a title produce.

[fallback capture] — the issue list.

Do: resolve the issues with these pre-scripted decisions (read them out as your answers; do not improvise under time pressure):

1. “`index.md` is not a note. Amend the format spec’s scope so the front-matter and heading rules apply only to files in `notes/`; `index.md` is governed only by the index-consistency rule R6.”
2. “The H1 must be identical to the front-matter `title`. The front-matter is authoritative: a conformance repair adjusts the heading, never the field.”
3. “Add a filename rule R7: a note’s filename is the slug of its title plus `.md` — lowercase the title, remove apostrophes, replace every remaining run of non-letter/digit characters with a single hyphen, and strip leading and trailing hyphens.”

If the agent surfaces *more* issues than the seeded three (likely, and welcome): resolve each in one sentence, or say “defer it — record it under a ‘Deferred questions’ heading in note-set-operations.md when you write it.” **Say:** a living spec has a place for known unknowns.

Do: prompt:

Good. Before any scaffolding: propose the exact amendments to note-format-spec.md and note-set-conops.md as a numbered list, and wait for my approval before editing either file.

Do: approve the amendments (this is CLAUDE.md working rule 2 running in front of the class). Exit plan mode / accept the plan so the agent can edit the two specs, bump versions (0.1 (draft) → 1.0.0 / 1.0), add changelog entries, and commit.

Do (shell): `git diff HEAD~1 -- note-format-spec.md`, then `git log --oneline`.

Observe & say: information just flowed *backwards* — from implementation planning into the specifications, before a single artifact exists. The spec did not survive contact with planning, and that is the method working, not failing (claims 1 and 5). The changelog entry is the evidence: “surfaced during implementation planning.” Also name what just happened to claim 4: Gap A was an inconsistency *between* two specifications, and it is now fixed in both.

If it goes differently: if a seeded gap is missed, one nudge each — “Walk through executing ‘add blank note’ step by step: what filename do you create?” (Gap D); “Is `index.md` a note? Check it against R1.” (Gap A); “Can a conformant note’s H1 differ from its title field?” (Gap B). If plan mode misbehaves or the

version’s UI differs, run the same prompt in normal mode with “Do not create or modify any files until I approve a plan” appended — the pedagogy (planning surfaces spec defects) is mode-independent.

Segment 3 — Scaffold: a third specification appears (~7 min)

Do: prompt:

Proceed: scaffold the note set — the notes directory and an `index.md` satisfying R6 — and write `note-set-operations.md` documenting the four operations with preconditions and postconditions. Do not create any notes yet.

Expected: `notes/` created, a skeleton `index.md`, a new `note-set-operations.md`, a self-reported conformance status, one commit.

Do (shell): `cat note-set-operations.md`.

Observe & say: three specifications now govern this repository, each an *abstraction* of something different — the ConOps abstracts purpose, the format spec abstracts every conformant note, the operations document abstracts the agent’s behavior (claims 2 and 4). The pre/postconditions are Design by Contract applied to agent operations. And per working rule 7, the document the agent just wrote is now *binding on the agent that wrote it*.

Optional, if ahead of schedule: prompt:

Cross-check `note-set-operations.md` against the ConOps and rules R1–R7: is there any operation that, followed exactly, can produce a violating state?

Say: a spec-to-spec conformance check, performed by the agent.

If it goes differently: if the agent seeds an example note anyway, have it removed via O3 — an unplanned early demo of remove, at zero cost.

Segment 4 — Operations: the invariant in motion (~8 min)

Do: prompt:

Add a blank note titled “Royce 1970 Waterfall Paper”.

Expected: `notes/royce-1970-waterfall-paper.md` — front-matter with an ISO date, H1 equal to the title, an index entry, a rule-citing conformance report, a commit. **Say:** that filename exists only because planning refined the spec an hour ago (R7).

Do: prompt:

Add a note titled “No Silver Bullet” with the initial content I’m about to paste. Bring it into conformance with the format spec, preserving my meaning.

then paste the contents of `sample-inputs/no-silver-bullet-content.md`.

Expected: the pasted `##` sections wrapped in front-matter and the correct H1, indexed, checked, committed.

Say: notice O2’s postcondition has a judgment clause — “preserving the user’s meaning.” Hold that phrase; it comes back in Segment 6.

Do: prompt:

Add a blank note titled “Design by Contract”, then remove the note titled “Design by Contract”.

Expected: add, then remove; the file gone *and* the index entry gone.

Do (shell): `ls notes/` and `git log --oneline`.

Observe & say: R6 — index consistency — held through every operation. Conformance is a continuously maintained invariant, not an end-of-project gate (claim 3). The commit trail names each operation: an audit log of the invariant being maintained.

If it goes differently: if the agent balks at the pointless add/remove pair, say why aloud — “we need O3’s precondition satisfied to demo O3” — and insist once.

Segment 5 — The verification gate (~11 min)

Do (editor, agent idle): open `notes/royce-1970-waterfall-paper.md` in the plain-text editor, select all, paste the entire contents of `sample-inputs/royce-note-filled-nonconformant.md` over it, save.

Say: “I filled this in myself last night, in another editor, the way real users actually work. No process was watching.”

For your eyes — the paste contains exactly four violations:

#	In the file	Rule	Repair
1	<code>created: Sept 11, 2026</code>	R2 (original, v0.1)	<code>2026-09-11</code>
2	<code>H1 # The Waterfall Paper vs. title: Royce 1970 Waterfall Paper</code>	R3 (added by Segment 2’s refinement)	H1 rewritten; front-matter authoritative
3	<code>a second H1, # My take</code>	R4	demote to <code>##</code>
4	<code>## What Royce actually said followed by ####</code>	R5	demote to <code>###</code>

Do: prompt:

I filled in the Royce note myself last night. Check it for conformance to `note-format-spec.md` and report what you find before fixing anything.

Expected: four findings citing R2, R3, R4, R5, each quoting the rule text (working rules 4 and 5), and the agent *waits*.

[fallback capture] — the four-finding report.

Observe & say: the R3 finding — H1 disagrees with title — is catchable *only because of Segment 2*. The draft spec would have blessed this file. And watch the tool calls: the agent Read the spec before judging. The check is driven by the document, not by the agent’s memory of it (claims 1, 2).

Do: prompt:

Repair the note to conform. Keep my wording except where conformance requires changes, then re-check and finish per the working rules.

Expected: the four mechanical repairs, a passing re-check, a commit.

Observe & say: the passing re-check *is* the completion gate — the task is not “edit the file,” it is “make the conformance predicate true” (claim 3). Bookend it: in Segment 2 the spec moved while the implementation held still; here the implementation moved while the spec held still — and in both cases the demo ends with conformance re-established.

If it goes differently: if the agent repairs without waiting, don’t fight it — `git diff HEAD~1` shows all four repairs; teach from the diff, then name the skipped working rule: “instructions influence, they don’t enforce — lecture 05’s point, recurring.”

Segment 6 — The algorithmic checker (~10 min)

Say first: that check burned tokens and could vary run to run. The S/R essay’s examples — a compiler, a state-machine generator — show conformance checking going *algorithmic*. The mechanical part of this spec deserves the same.

Do: prompt:

Conformance checking shouldn't need you every time. Write a Python 3 script, `check_notes.py`, `stdlib` only, that checks the mechanical rules of `note-format-spec.md`. It takes one or more note paths, or `--all` to check every note plus the index rule R6. Print one line per violation citing the rule ID; exit 0 if conformant, 1 if there are violations, 2 on usage errors. Then update `note-set-operations.md`: record which rules the script checks and which still require agent judgment, and make the check-conformance operation run the script first.

Expected: the script written and run (`--all` passes on the current set), the operations doc gains a mechanical-vs-judgment table, a commit.

Do (shell): sabotage live — in the editor, change the Royce note's `created:` date to `Sept 2026` and save, then:

```
python3 check_notes.py --all; echo "exit: $?"
git checkout -- notes/royce-1970-waterfall-paper.md
python3 check_notes.py --all; echo "exit: $?"
```

Expected: one R2 line and exit 1; after restore, all conformant and exit 0.

[fallback capture] — both runs.

Observe & say: deterministic, instant, token-free — and it cites the *same rule IDs* as the agent's Segment 5 report, because the spec numbered its rules to be script-generable. The exit code makes it composable: a pre-commit hook, CI, or — once we cover them — a skill. One sentence on the parser: the spec restricts front-matter to two scalar fields, so a fifteen-line `stdlib` parser is a *complete* parser for the specified language — the spec was tightened until the checker could be trivial (checkability flowing back into the spec; a real PKB would justify PyYAML). Then the contrast that is the segment's point: the script can check that a section *exists*; it can never check that a repair “preserved my meaning.” Mechanically checkable clauses vs. judgment clauses — the script covers the first set; the agent remains the instrument for the second. Finally: this script is itself a realization of the format spec. Hold that thought for five minutes.

If it goes differently: if the live-built checker has a bug, time-box debugging to two minutes and frame it honestly — “the checker is a realization that failed conformance; the same loop applies to it.” Then run the rehearsed reference from outside the repo: `python3 ~/note-set-demo-fallback/check_notes.py --all`. (Note: the reference checker is v2-aware; on the raw Segment 5 paste it reports a fifth finding, R8, that the live v1 checker would not — that discrepancy is only visible if you run it against pre-migration files, and it is itself explainable: the checker tracks the spec's version.)

Segment 7 — Spec v2 and the migration (~10 min)

Say: requirements change. “I now want the index to tell me what each note *says*, not just that it exists.”

Do: prompt:

New requirement: every note must begin with a “`### Summary`” section — one short paragraph, the first section after the title — and each index entry must show the first sentence of that summary. Treat this as a specification change: propose the amendments to `note-format-spec.md` (a new rule R8, an amended R6, a version bump and changelog entry), plus the matching updates to `note-set-operations.md`, `check_notes.py`, the two existing notes, and `index.md`. Show me the spec amendments first and wait for approval.

Do: approve the amendments.

Expected: the spec goes to 2.0.0 with a breaking-change changelog entry; the checker gains R8 and the index-entry check; and the agent **writes summaries** for the Brooks and Royce notes — watch it re-read each note first: this is judgment work, not templating. Index rebuilt with summary sentences, `--all` passes, one migration commit.

[fallback capture] — the two freshly written summaries.

Observe & say: one requirement, and everything co-evolved in a single coordinated change — specification, checker, realizations, index — with the conformance invariant re-established at v2 (claims 1, 2, 3). The summaries are the judgment clause incarnate: R8’s MUST half (a Summary section exists, one paragraph) landed in the checker; its SHOULD half (the summary is *accurate*) only the agent or you can judge — the operations doc’s table now shows R8 split across both columns. And the held thought from Segment 6: had the checker stayed at v1, it would now be a *false green light* — that is exactly why the prompt bundled spec, checker, notes, and index into one change.

Optional, if ahead of schedule: prompt:

Add a note titled “Lehman’s Laws of Software Evolution” with the initial content I’m about to paste.

then paste `sample-inputs/lehmans-laws-content.md`. **Expected:** born v2-conformant — Summary written unprompted, and the apostrophe in the title exercises R7 (`lehmans-laws-of-software-evolution.md`). **Say:** Lehman’s Law of Continuing Change is, verbatim, what this segment just enacted — on a specification.

If it goes differently: if the agent migrates before showing amendments, stop it once: “amendments first, per working rule 2.” If a summary is weak, improve it by editing the *spec* — tighten R8’s SHOULD clause live and re-migrate: even the closing segment can refine a spec.

Segment 8 — Wrap: five claims, five pieces of evidence (~5 min)

Say: walk the table.

Claim	Evidence from the last hour
A spec is a continuously maintained document	v0.1 → 1.0.0 → 2.0.0 under git, with changelogs and a Deferred-questions section
A spec is an abstraction of the implementation	R1–R8 summarize every conformant note; the ops doc abstracts the agent’s behavior; the v2 index literally displays abstractions
Conformance is an invariant while both sides evolve	Segment 5 (implementation moved, spec still); Segments 2 and 7 (spec moved, implementation migrated); a gate on every operation
Multiple specs, mutually consistent	ConOps / format spec / operations doc; Gap A was an inter-spec inconsistency, found and fixed
Information flows both ways	planning produced R7 and the R3 amendment; checkability shaped the spec; a new requirement co-evolved four artifacts

Say (the closing irony): the note sitting in our own set — Royce 1970 — warned in the founding paper of “waterfall” that single-pass development “is risky and invites failure.” The demo’s first artifact contained its own thesis.

Say (the bridge): “Project 0 is this, at your scale. Your PKB spec is these three documents grown up; its conformance checklist is R1–R8 grown up; the stretch-goal validator is `check_notes.py` grown up. Specs are portable; prompts are not — everything that mattered today lives in files, not in anything I typed.”

Recreating this demo yourself (students)

Copy `starter/` somewhere outside the course repo, follow the setup checklist, and run Segments 2–7 with your own Claude Code session. Your run **will** differ: a different gap list in Segment 2, different summary wording in Segment 7, possibly a different slug convention if you decide Gap D differently. That is the exercise — compare your final `note-format-spec.md` against `completed/` and you are looking at two realizations of the same intent, which is the S/R essay’s central point. When a beat diverges from this script, read

the segment's *If it goes differently* note; the mismatch is usually the interesting part. Do not peek at `sample-inputs/royce-note-filled-nonconformant.md` before running your own Segment 5 — hand your agent a note *you* wrote carelessly instead, and see whether your spec catches what you got wrong.